

Redes Neuronales Convolucionales

Óscar Climent Ivorra

Trabajo Fin de Grado
Grado en Matemáticas

11 de junio de 2026

Motivación y objetivo del trabajo

Motivación del trabajo

La red neuronal convolucional es una herramienta eficaz para la clasificación de imágenes.

Objetivo del trabajo

El objetivo principal del Trabajo Fin de Grado es realizar una introducción a las **redes neuronales convolucionales** y a su aplicación a la clasificación de imágenes.

Estructura de la exposición

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Índice

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Problemas de predicción

En muchas aplicaciones de interés, el objetivo es resolver un **problema de predicción**: a partir de *input* (datos de entrada) x , se quiere predecir un *output* (una salida) y .

Tipos de problemas

- ▶ **Regresión**: la salida es una variable cuantitativa.
- ▶ **Clasificación**: la salida es una variable cualitativa

En este trabajo nos centramos en los problemas de clasificación.

Se necesita definir una función de predicción $f(\cdot)$, que asocia un input x un output $f(x)$.

Aprendizaje automático

Proporciona algoritmos para construir funciones de predicciones con buenas condiciones y cuantificar el rendimiento.

Funcionamiento de los algoritmos

- ▶ Se define **conjunto de entrenamiento** y **conjunto test**.
- ▶ Se calcula el rendimiento de la función de predicción mediante el error de predicción.

El proceso de aprendizaje de la función de predicción recibe el nombre de aprendizaje supervisado.

Índice

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Redes neuronales

Definición de Red Neuronal

Una red neuronal permite construir, a partir de un conjunto de datos de entrenamiento, una función de predicción no lineal que asocia a un *input* x un *output* y .

Datos de entrada: $\mathbf{X} = (X_1, X_2, \dots, X_p) \in \mathbb{R}^p$

Variable de salida:

- **Problema de clasificación con C clases:** la variable de salida se representa mediante un vector binario

$$Y = (Y_1, \dots, Y_C)$$

de dimensión C , usando la codificación *1-de-K*.

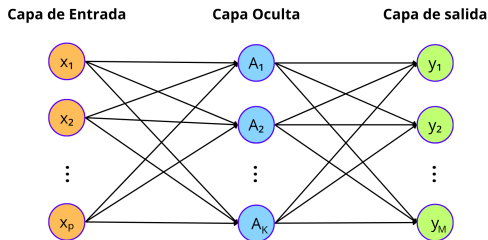
- Si la clase verdadera es $h \in \{1, \dots, C\}$, entonces:

$$Y_c = \begin{cases} 1, & \text{si } c = h, \\ 0, & \text{si } c \neq h, \end{cases} \quad c = 1, \dots, C.$$

Estructura de una red neuronal

Las redes neuronales se basan en el funcionamiento del cerebro humano. Estas están formadas por capas de neuronas:

- ▶ **Capa de entrada:** Recibe datos de entrada y contiene p neuronas.
- ▶ **Capas ocultas:** Procesan información de la capa de entrada.
- ▶ **Capa de salida:** Devuelve el output.
 - Problema de clasificación de C clases: C neuronas.

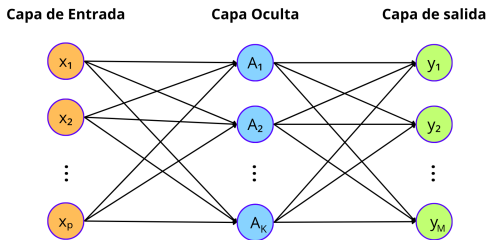


Redes neuronales prealimentadas (FNN)

Redes neuronales prealimentadas (FNN)

Son las redes más sencillas, donde la información fluye en una única dirección. Tienen como caso particular las redes convolucionales.

Suponemos primero el caso más sencillo de FNN: una capa oculta de K neuronas y una capa de salida con M neuronas.



Funcionamiento de una neurona de la capa oculta

Preactivación y activación de una neurona de la capa oculta

Recibe como datos de entrada el vector de *input*.

$$a_k = w_{k0} + \sum_{j=1}^p w_{kj} X_j, \quad k = 1, \dots, K$$

$$A_k = g(a_k), \quad k = 1, \dots, K$$

donde w_{k0} es el sesgo, w_{kj} es el peso asociado a la conexión de la neurona j a la neurona k y $g(\cdot)$ es la función de activación no lineal diferenciable.

Funcionamiento de las neuronas de la capa de salida

En este caso, las neuronas de la capa de salida reciben las activaciones de las neuronas de la capa oculta.

Preactivación y activación de las neuronas de la capa de salida

$$a_h^{out} = w_{h0}^{out} + \sum_{k=1}^K w_{hk}^{out} \cdot A_k, \quad h = 1, \dots, M$$

$$A_h^{out} = f(a_h^{out}), \quad h = 1, \dots, M$$

donde w_{hk}^{out} es el peso asociado a la conexión de la neurona k de la capa oculta y la neurona h de la capa de salida, w_{h0}^{out} es el sesgo de la neurona h de la capa de salida y f es la función de activación de la neurona h .

Redes neuronales prealimentadas multicapa

Supongamos que tenemos una red neuronal prealimentada con $L \geq 2$ capas ocultas, la notación anterior se puede generalizar.

Preactivación y activación de las neuronas de la $l \in \{2, \dots, L\}$ -ésima capa oculta

$$a_k^{(l)} = w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} \cdot A_j^{(l-1)}, \quad k = 1, \dots, K_l$$

$$A_k^{(l)} = g_l(a_k^{(l)}), \quad k = 1, \dots, K_l$$

Redes neuronales prealimentadas multicapa

Preactivación y activación de las neuronas de la capa de salida

$$a_h^{out} = w_{h0}^{out} + \sum_{k=1}^{K_L} w_{hk}^{out} \cdot A_k^{(L)}, \quad h = 1, \dots, M$$

$$y_h^{out} = A_h^{out} = f(a_h^{out}), \quad h = 1, \dots, M$$

Función de error y entrenamiento

Una vez fijada la arquitectura, la red puede verse como un modelo de predicción paramétrico, con vector de parámetros $\mathbf{w}$, que asocia a cada *input* $\mathbf{X}$ un output $f(\mathbf{X}, \mathbf{w}) = (A_1^{out}(\mathbf{X}, \mathbf{w}), \dots, A_M^{out}(\mathbf{X}, \mathbf{w}))$

Error de entrenamiento

$$R(\mathbf{w}) = \sum_{i=1}^{N_{\text{train}}} L(f(X_i, \mathbf{w}), Y_i).$$

donde L es la función de error que cuantifica el error de predicción para dicho individuo.

La estimación de $\mathbf{w}$ se obtiene resolviendo el problema

$$(P) \quad \min_{\mathbf{w}} R(\mathbf{w}).$$

Entrenamiento de las redes neuronales

El proceso de aprendizaje se basa en ir ajustando los pesos a partir de la experiencia.

Para determinar el vector de parámetros $\mathbf{w}$, se utilizan diferentes métodos:

- ▶ **Descenso del gradiente (GD)**
- ▶ **Descenso del gradiente estocástico (SGD)**
- ▶ **Descenso del gradiente por mini-lotes (mini-batch SGD)**

Por otro lado, en dichos métodos se utiliza $\nabla R(\mathbf{w})$. Por ello, para calcularlo de una forma eficaz, utilizaremos el **método de retropropagación**.

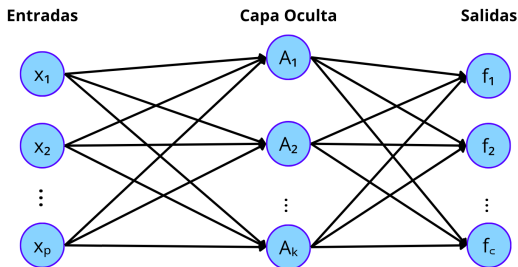
Método de retropropagación

- ▶ Es un algoritmo muy utilizado en el ámbito de las redes neuronales.
- ▶ La retropropagación permite calcular eficientemente las derivadas parciales del error de entrenamiento para una observación del conjunto de entrenamiento, $\nabla R_n(w)$.
- ▶ El cálculo de $\nabla R_n(w)$ consta de 3 fases:

Método de retropropagación

Fases

- 1 **Propagación hacia delante:** Cálculo preactivaciones y activaciones de las neurona y predicción de la red
- 2 **Cálculo del error:** Calculo de $R_n(\mathbf{w})$ y $\frac{\partial R_n}{\partial a_c^{\text{out}}}$, $c = 1, \dots, C$
- 3 **Propagación hacia atrás:** Cálculo del término de error de las neuronas y ∇R_n .



Índice

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)**
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son diseñadas para trabajar con datos que presentan una estructura espacial, como las imágenes.

Las CNN utiliza la información de la que se dispone sobre la estructura de los datos de entrada para diseñar la arquitectura del modelo de red neuronal.

La CNN deberá identificar un objetivo en una imagen y su posición. Además, la jerarquía que presentan es muy importante.

Estructura de las redes neuronales convolucionales

La red neuronal convolucional está formada por la combinación sucesiva de dos capas ocultas:

- ▶ Capas de convolución
- ▶ Capas de agrupación

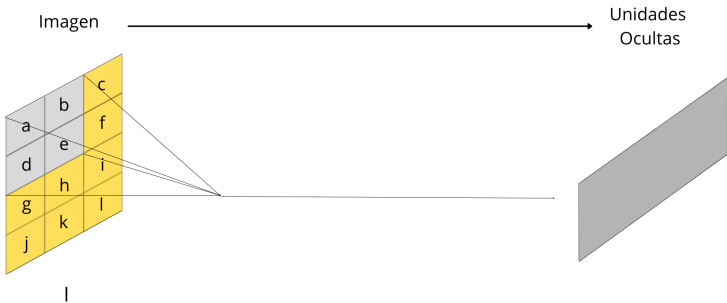
Capa de convolución

Dada una imagen de entrada (en escala de grises), I , de dimensión $J \times H$, donde $I(j, h)$ denotamos por $I(j, h)$ la intensidad del píxel (j, h) , donde $j = 1, \dots, J$, $h = 1, \dots, H$.

Capa convolucional y Campo receptivo

Una unidad de la primera capa oculta, llamada **capa convolucional**, toma como entrada las intensidades de los píxeles de una región pequeña de la imagen, llamada **campo receptivo**, de dimensión $M \times M$ con $M < \min\{J, H\}$.

Detección de características



Detección de características

Filtro

Los píxeles que forman el campo receptivo tienen asociado un peso, y estos forman un **filtro**, K , de la misma dimensión que el campo receptivo.

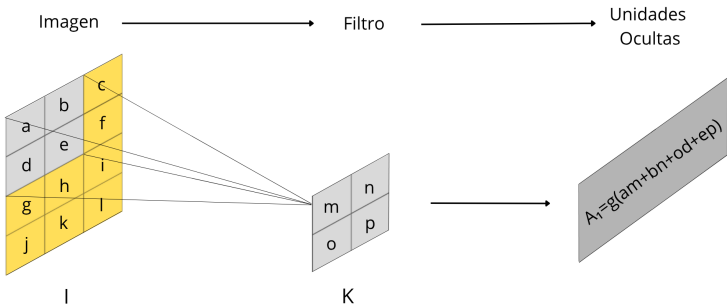
Preactivación y activación de una neurona

$$a = \mathbf{w}^t \cdot \mathbf{x} + w_0$$

$$A = g(\mathbf{w}^t \cdot \mathbf{x} + w_0)$$

donde $\mathbf{x}$ son los píxeles del campo receptivo, $\mathbf{w}$ vector de pesos que definen el filtro, w_0 el sesgo y $g(\cdot)$ la función de activación de la unidad.

Detección de características



Interpretación

Los pesos del campo receptivo permiten capturar características locales de bajo nivel de la imagen.

Equivarianza a traslaciones

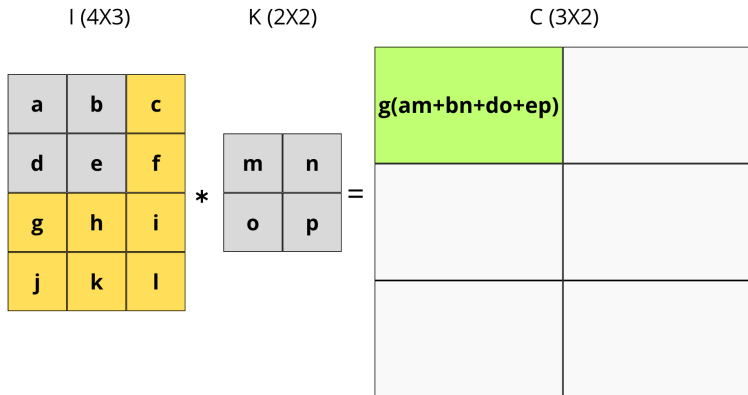
La red debe ser capaz de detectar la presencia de ciertas características en una región de la imagen y poder reconocerla en otras posiciones sin necesidad de aprenderla nuevamente en otra posición.

Fijando pesos del filtro y desplazando el campo receptivo se obtienen las neuronas de la capa convolucional.

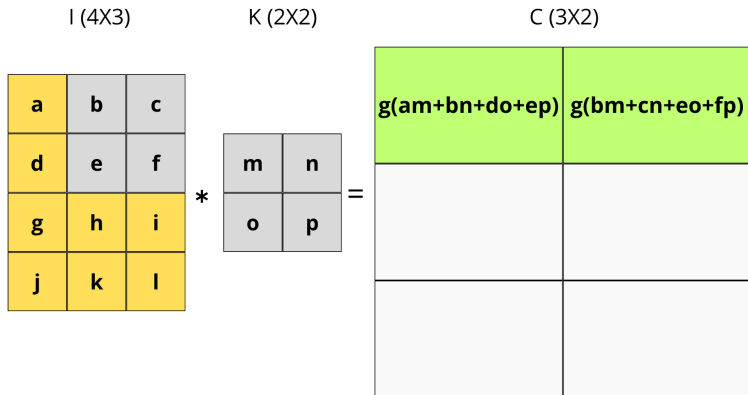
Número de desplazamientos del campo receptivo = Número de neuronas en la capa oculta.

La capa convolucional se ve como una imagen, llamada **mapa de características**, C , y se obtiene mediante la **convolución**.

Ejemplo Convolución: Paso 1



Ejemplo Convolución: Paso 2



Ejemplo Convolución: Paso 3

$$\begin{array}{c} \text{I (4X3)} \\ \begin{array}{|c|c|c|} \hline a & b & c \\ \hline d & e & f \\ \hline g & h & i \\ \hline j & k & l \\ \hline \end{array} \end{array} * \begin{array}{c} \text{K (2X2)} \\ \begin{array}{|c|c|} \hline m & n \\ \hline o & p \\ \hline \end{array} \end{array} = \begin{array}{c} \text{C (3X2)} \\ \begin{array}{|c|c|} \hline g(am+bn+do+ep) & g(bm+cn+eo+fp) \\ \hline g(dm+en+go+hp) & \\ \hline & \\ \hline \end{array} \end{array}$$

Ejemplo Convolución: Paso 4

$$\begin{array}{c} \text{I (4X3)} \\ \begin{array}{|c|c|c|} \hline a & b & c \\ \hline d & e & f \\ \hline g & h & i \\ \hline j & k & l \\ \hline \end{array} \end{array} * \begin{array}{c} \text{K (2X2)} \\ \begin{array}{|c|c|} \hline m & n \\ \hline o & p \\ \hline \end{array} \end{array} = \begin{array}{c} \text{C (3X2)} \\ \begin{array}{|c|c|} \hline g(am+bn+do+ep) & g(bm+cn+eo+fp) \\ \hline g(dm+en+go+hp) & g(em+fn+ho+ip) \\ \hline \end{array} \end{array}$$

Ejemplo Convolución: Paso 5

$$\begin{array}{c} \text{I (4X3)} \\ \begin{array}{|c|c|c|} \hline a & b & c \\ \hline d & e & f \\ \hline g & h & i \\ \hline j & k & l \\ \hline \end{array} \end{array} * \begin{array}{c} \text{K (2X2)} \\ \begin{array}{|c|c|} \hline m & n \\ \hline o & p \\ \hline \end{array} \end{array} = \begin{array}{c} \text{C (3X2)} \\ \begin{array}{|c|c|} \hline g(am+bn+do+ep) & g(bm+cn+eo+fp) \\ \hline g(dm+en+go+hp) & g(em+fn+ho+ip) \\ \hline g(gm+hn+jo+kp) & \\ \hline \end{array} \end{array}$$

Ejemplo Convolución: Paso 6

$$\begin{array}{c} \text{I (4X3)} \\ \begin{array}{|c|c|c|} \hline a & b & c \\ \hline d & e & f \\ \hline g & h & i \\ \hline j & k & l \\ \hline \end{array} \end{array} * \begin{array}{c} \text{K (2X2)} \\ \begin{array}{|c|c|} \hline m & n \\ \hline o & p \\ \hline \end{array} \end{array} = \begin{array}{c} \text{C (3X2)} \\ \begin{array}{|c|c|} \hline g(am+bn+do+ep) & g(bm+cn+eo+fp) \\ \hline g(dm+en+go+hp) & g(em+fn+ho+ip) \\ \hline g(gm+hn+jo+kp) & g(hm+in+ko+lp) \\ \hline \end{array} \end{array}$$

Operación de convolución

Operación de convolución

$$C(j, k) = g \left(\sum_{l=1}^L \sum_{m=1}^M I(j+l-1, k+m-1) K(l, m) \right)$$

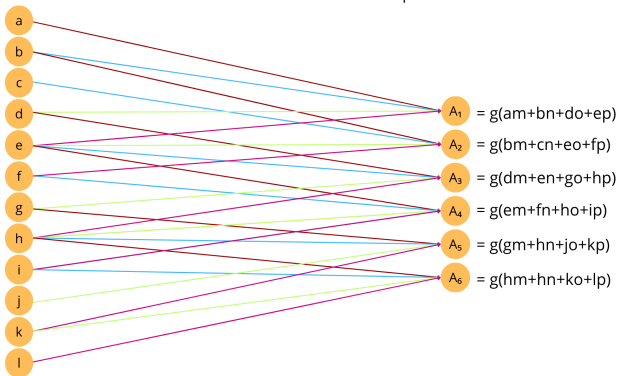
$$\text{con } j \in \{1, \dots, J-L+1\}, \quad k \in \{1, \dots, H-M+1\}$$

donde I es la imagen de entrada de dimensión $J \times H$, K es el filtro de dimensión $L \times M$ y $g(\cdot)$ es la función de activación de la unidad.

De la imagen a las unidades ocultas

Capa de entrada

Capa convolucional



Equivarianza

Con este proceso se formaliza el concepto de equivarianza, por la conexiones locales y la compartición de pesos.

Capa de agrupación

Si el coste computacional es muy grande, hay técnicas para poder reducirlo:

- ▶ **Stride:** Permite desplazar el filtro de forma continua pero con salto.
- ▶ **Agrupación:** Consiste en desplazar un filtro por el mapa de características aplicando una función simple y fija.

Capa de Agrupación

Se llama capa de agrupación a la capa oculta que utiliza la técnica de agrupación, reduciendo el mapa de características e introduciendo invarianza local respecto a pequeñas traslaciones.

Índice

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Conjunto de datos MNIST

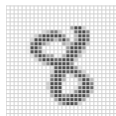
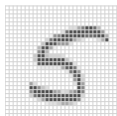
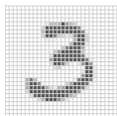
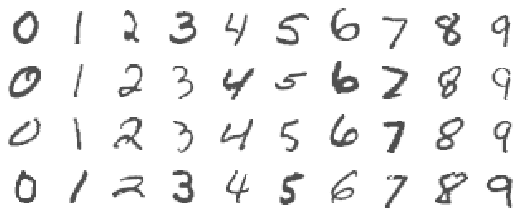
MNIST es un conjunto de datos que contiene 70.000 imágenes en escala de grises de tamaño 28×28 . Cada imagen representa un dígito manuscrito del 0 al 9.

Conjunto de datos MNIST

Las 70.000 imágenes se dividen en:

- ▶ Conjunto de entrenamiento = 60.000 imágenes (48.000 para el entrenamiento y 12.000 para la validación)
- ▶ Conjunto test = 10.000 imágenes

Ejemplo de imágenes MNIST



Características de las redes implementadas

Tasa de acierto

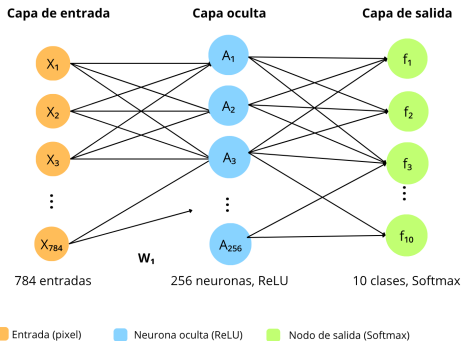
Para cuantificar el rendimiento de la red neuronal en el conjunto test se ha utilizado la tasa de acierto.

$$\text{Tasa de acierto} = \frac{\text{número de predicciones correctas}}{\text{número total de observaciones}} \times 100$$

Épocas de entrenamiento

En las redes implementadas se han utilizado 30 épocas.

FNN de una única capa oculta implementada



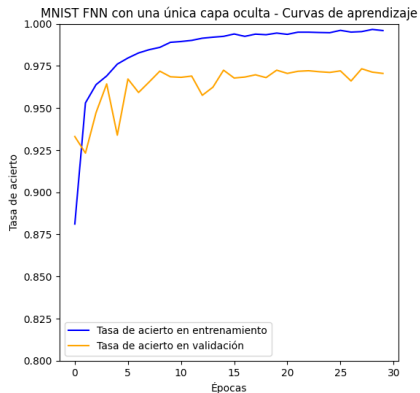
Observación

No se han obtenido mejores resultados con más capas ocultas.

Resultado

La FNN obtiene una tasa de acierto del **97,29 %**.

Curvas de aprendizaje de la FNN

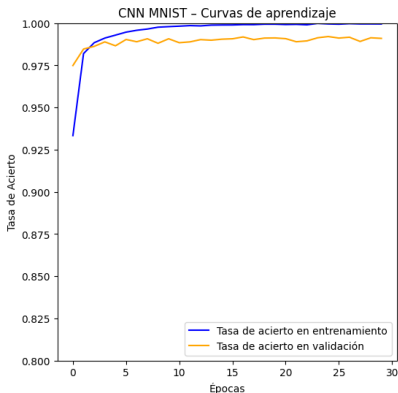


Observación

Se observa un sobreajuste en el modelo y alrededor la época 15 se estabiliza la tasa de acierto en validación.

Modelo CNN implementado

Formada por 2 capas convolucionales, la primera y la tercera, seguidas cada una de una capa de agrupación por máximo y una última capa totalmente conectada.



Resultado

La CNN obtiene una tasa de acierto del **99,05 %**.

Observación

Se observa un sobreajuste menor en el modelo y alrededor de las primeras épocas se estabiliza la tasa de acierto en validación.

Comparación de resultados

Modelo	Tasa de acierto
FNN	97,29 %
CNN	99,05 %

Conclusión experimental

La CNN obtiene un rendimiento superior ya que aprovecha la estructura espacial de las imágenes de entrada, mientras que la FNN aplanar los píxeles de la imagen de entrada en un vector perdiendo información espacial.

Índice

- 1 Introducción
- 2 Redes Neuronales
 - Redes Neuronales Prealimentadas (FNN)
 - Entrenamiento de las redes neuronales
- 3 Redes Neuronales Convolucionales (CNN)
 - Capa de convolución
 - Capa de agrupación
- 4 Ejemplo Práctico MNIST con una FNN y CNN
- 5 Conclusiones

Conclusiones del trabajo

- ▶ Presentación de la arquitectura de las redes neuronales prealimentadas con una única capa oculta, generalizando el modelo multicapa y la explicación la base teorica del entrenamiento en redes neuronales
- ▶ Se ha realizado una explicación teórica de la arquitectura de las redes neuronales convolucionales, junto con las capas de convolución y agrupación.
- ▶ Se ha ilustrado las ventajas de las redes neuronales convoluciones con respecto a redes prealimentadas estándar en un problema de clasificación de imagen multiclase, utilizando el conjunto de datos MNIST.

Posibles trabajos futuros

- ▶ Desarrollar la teoría de las técnicas de regularización para evitar un sobreajuste de la red.
- ▶ Abordar otras importantes aplicaciones de las CNN:
 - La generación automática de un texto que describe el entorno de una imagen.
 - Generación de imágenes basadas en un *input* de texto que describe el contenido de la misma.
 - Etc.

Fin

Gracias por su atención